Langage C

Résumé complet des notions
+ fonctions récurrentes des exercices

Partie 1	Notions du langage C (syntaxe, pointeurs, mémoire, fichiers...)
Partie 2	Fonctions qu'on réécrit dans presque tous les exercices (tableaux, tris, maths, chaînes, matrices, structures, listes chaînées, fichiers)
Partie 3	Modèle de programme (menu) et pièges à éviter

PARTIE 1 - Notions du langage C

1. Structure d'un programme

```
#include <stdio.h>

int main(void) {
    printf("Bonjour\n");
    return 0;
}
```

- `#include` : inclut un en-tête (bibliothèque). `main` : point d'entrée, retourne `0` si tout va bien.
- Chaque instruction se termine par `;`. Commentaires : `// ligne` et `/* bloc */`.
- **Compilation** : `gcc fichier.c -o programme -Wall -Wextra`, puis `./programme`.
- Étapes : préprocesseur, compilation, assemblage, édition de liens.

2. Types et variables

Type	Taille typique	Exemple
`char`	1 octet	`'a'`
`short` / `int` / `long` / `long long`	2 / 4 / 8 / 8 octets	`42`
`float` / `double`	4 / 8 octets	`3.14`
`unsigned`	entiers positifs seulement	`unsigned int x;`

- Déclaration : `int x = 5;`. Constantes : `const int N = 10;` ou `#define N 10`.
- `sizeof(type)` donne la taille en octets. `<stdint.h>` : `int8_t`, `uint32_t`... `<stdbool.h>` : `bool`.
- Cast (conversion explicite) : `(float)a / b`.

3. Entrées / sorties

```
int n; float f; char s[50];
scanf("%d", &n);              // & obligatoire (sauf tableaux/chaînes)
printf("n = %d\n", n);
fgets(s, sizeof(s), stdin);   // lecture sûre d'une ligne
```

Formats : `%d` int, `%u` unsigned, `%ld` long, `%f` float (printf), `%lf` double (scanf), `%c` char, `%s` chaîne, `%p` pointeur, `%x` hexadécimal, `%.2f` 2 décimales.

4. Opérateurs

- Arithmétiques : `+ - * / %`. Comparaison : `== != < > <= >=`. Logiques : `&& || !`.
- Bit à bit : `& | ^ ~ << >>`. Affectation : `= += -= *= /= %=`. Incrément : `i++`, `++i`, `i--`, `--i`.
- Ternaire : `cond ? a : b`. Attention : la division de deux entiers est entière (`5/2 = 2`).

5. Structures de contrôle

```
if (x > 0) { ... } else if (x == 0) { ... } else { ... }
switch (c) { case 1: ...; break;  default: ...; }
for (int i = 0; i < n; i++) { ... }
while (cond) { ... }
do { ... } while (cond);          // s'exécute au moins une fois
```

`break` quitte la boucle, `continue` passe à l'itération suivante. `goto` : déconseillé.

6. Fonctions

```
int somme(int a, int b);        // prototype (avant main)
int somme(int a, int b) {       // définition
    return a + b;
}
```

- Passage **par valeur** par défaut. Pour modifier l'original : passer un pointeur.
- `void` : pas de retour / pas de paramètre. Récursivité : toujours un cas d'arrêt.
- `static` local : la variable garde sa valeur entre les appels. Éviter les variables globales.

7. Tableaux

```
int t[5] = {1, 2, 3, 4, 5};
int m[3][3];                    // tableau 2D
t[0] = 10;                      // indices de 0 à n-1
```

- Aucun contrôle des bornes : dépassement = comportement indéfini.
- Un tableau passé à une fonction devient un pointeur : il faut passer sa taille.

8. Chaînes de caractères

Tableau de `char` terminé par `'\0'` : `char s[] = "hello";` occupe 6 octets.

- `<string.h>` : `strlen`, `strcpy`, `strncpy`, `strcat`, `strcmp`, `strchr`, `strstr`.
- `<ctype.h>` : `toupper`, `tolower`, `isdigit`, `isalpha`, `isspace`. `<stdlib.h>` : `atoi`, `atof`, `strtol`.
- Comparer avec `strcmp` (retourne 0 si égales), jamais avec `==`.

9. Pointeurs

```
int x = 10;
int *p = &x;      // p contient l'adresse de x
*p = 20;          // déréférencement : x vaut 20
```

- `&` : adresse de. `*` : valeur pointée. `NULL` : pointeur nul (tester avant de déréférencer).
- Arithmétique : `p + 1` avance d'un élément. `t[i]` équivaut à `*(t + i)`.
- Pointeur de pointeur `int **pp`. Pointeur de fonction : `int (*f)(int, int) = somme;`
- `const int *p` (valeur non modifiable) et `int *const p` (adresse non modifiable) sont différents.
- `void *` : pointeur générique.

10. Structures, unions, énumérations

```
typedef struct { char nom[30]; int age; } Personne;
Personne p = {"Ali", 20};
p.age = 21;                  // accès avec .
Personne *pp = &p;
pp->age = 22;                // accès via pointeur avec ->

union Val { int i; float f; };      // membres partagent la même mémoire
enum Jour { LUN, MAR, MER };        // constantes 0, 1, 2...
```

11. Allocation dynamique

```
int *t = malloc(n * sizeof(int));
if (t == NULL) { /* erreur */ }
t = realloc(t, 2 * n * sizeof(int));
free(t);  t = NULL;
```

- `malloc` : non initialisé. `calloc(n, taille)` : initialisé à 0. Chaque `malloc` doit avoir son `free`.
- Erreurs fréquentes : double `free`, utilisation après `free`, dépassement de tampon, fuite mémoire.
- **Pile** : variables locales (automatique). **Tas** : allocation dynamique (manuelle). Outil : `valgrind`.

12. Fichiers

```
FILE *f = fopen("data.txt", "r");        // "r" "w" "a" "rb" "wb"
if (f == NULL) { perror("fopen"); return 1; }
fprintf(f, "..."); fscanf(f, "%d", &n);  fgets(buf, sizeof(buf), f);
fread(...);  fwrite(...);                // mode binaire
fclose(f);
```

Utiles : `fseek`, `ftell`, `rewind`, `feof`.

13. Préprocesseur et programmation modulaire

```
#define PI 3.14159
#define CARRE(x) ((x)*(x))       // parenthèses importantes

#ifndef MON_HEADER_H             // garde d'inclusion
#define MON_HEADER_H
```

```
...
#endif
```

- `fichier.h` : déclarations (prototypes, types). `fichier.c` : définitions.
- Compilation séparée : `gcc -c a.c`, `gcc -c b.c`, `gcc a.o b.o -o prog`. `Makefile` pour automatiser.
- `static` global : visible seulement dans le fichier. `extern` : variable définie ailleurs.
- Arguments de `main` : `int main(int argc, char *argv[])` ; `argv[0]` = nom du programme.

14. Bibliothèque standard essentielle

- `<stdio.h>` : entrées/sorties. `<stdlib.h>` : `malloc`, `rand`, `srand`, `exit`, `qsort`, `atoi`, `abs`.
- `<string.h>` : chaînes, `memcpy`, `memset`. `<math.h>` : `sqrt`, `pow`, `sin`, `fabs` (compiler avec `-lm`).
- `<time.h>` : `time`, `clock`. `<ctype.h>`, `<limits.h>`, `<stdbool.h>`.

PARTIE 2 - Fonctions récurrentes des exercices

Ces fonctions reviennent dans presque tous les exercices (TD, examens). Il faut savoir les écrire de tête. Les matrices utilisent des tableaux de taille variable (norme C99). Les en-têtes nécessaires : `stdio.h`, `stdlib.h`, `string.h`, `ctype.h`, `math.h`, `time.h`.

A. Utilitaires de base

```
void vider_buffer(void) {                // après scanf, avant fgets/getchar
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

int saisir_positif(void) {               // saisie contrôlée
    int n;
    do {
        printf("Donner n > 0 : ");
        scanf("%d", &n);
    } while (n <= 0);
    return n;
}

void echanger(int *a, int *b) {          // swap
    int t = *a; *a = *b; *b = t;
}

int aleatoire(int min, int max) {        // appeler srand(time(NULL)) une fois dans main
    return rand() % (max - min + 1) + min;
}
```

B. Tableaux à une dimension

```
void saisir_tab(int t[], int n) {
    for (int i = 0; i < n; i++) {
        printf("t[%d] = ", i);
        scanf("%d", &t[i]);
    }
}

void afficher_tab(const int t[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", t[i]);
    printf("\n");
}

void remplir_aleatoire(int t[], int n, int min, int max) {
    for (int i = 0; i < n; i++) t[i] = aleatoire(min, max);
}

int somme_tab(const int t[], int n) {
    int s = 0;
    for (int i = 0; i < n; i++) s += t[i];
    return s;
}

float moyenne_tab(const int t[], int n) { return (float)somme_tab(t, n) / n; }

int max_tab(const int t[], int n) {
    int m = t[0];
    for (int i = 1; i < n; i++) if (t[i] > m) m = t[i];
    return m;
}

int indice_min(const int t[], int n) {    // indice du minimum
    int im = 0;
    for (int i = 1; i < n; i++) if (t[i] < t[im]) im = i;
    return im;
}

int compter(const int t[], int n, int x) {    // nombre d'occurrences de x
    int c = 0;
    for (int i = 0; i < n; i++) if (t[i] == x) c++;
    return c;
}
```

```c
void inverser_tab(int t[], int n) {
    for (int i = 0; i < n / 2; i++) echanger(&t[i], &t[n - 1 - i]);
}

void copier_tab(const int src[], int dst[], int n) {
    for (int i = 0; i < n; i++) dst[i] = src[i];
}

int est_trie(const int t[], int n) {
    for (int i = 0; i < n - 1; i++) if (t[i] > t[i + 1]) return 0;
    return 1;
}

int supprimer_pos(int t[], int n, int pos) {  // retourne la nouvelle taille
    for (int i = pos; i < n - 1; i++) t[i] = t[i + 1];
    return n - 1;
}

int inserer_pos(int t[], int n, int pos, int x) {   // t doit avoir la place
    for (int i = n; i > pos; i--) t[i] = t[i - 1];
    t[pos] = x;
    return n + 1;
}
```

C. Recherche

```c
int recherche_seq(const int t[], int n, int x) {    // -1 si absent
    for (int i = 0; i < n; i++) if (t[i] == x) return i;
    return -1;
}

int recherche_dicho(const int t[], int n, int x) {  // tableau TRIÉ obligatoire
    int g = 0, d = n - 1;
    while (g <= d) {
        int m = (g + d) / 2;
        if (t[m] == x) return m;
        if (t[m] < x) g = m + 1; else d = m - 1;
    }
    return -1;
}
```

D. Tris

```c
void tri_bulles(int t[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (t[j] > t[j + 1]) echanger(&t[j], &t[j + 1]);
}

void tri_selection(int t[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int im = i;
        for (int j = i + 1; j < n; j++) if (t[j] < t[im]) im = j;
        if (im != i) echanger(&t[i], &t[im]);
    }
}

void tri_insertion(int t[], int n) {
    for (int i = 1; i < n; i++) {
        int x = t[i], j = i - 1;
        while (j >= 0 && t[j] > x) { t[j + 1] = t[j]; j--; }
        t[j + 1] = x;
    }
}

int cmp_int(const void *a, const void *b) {          // pour qsort
    return *(const int *)a - *(const int *)b;
}
// utilisation : qsort(t, n, sizeof(int), cmp_int);
```

E. Fonctions mathématiques

```c
long factorielle(int n) {
    long f = 1;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
```

```
}

long puissance(int x, int n) {
    long r = 1;
    for (int i = 0; i < n; i++) r *= x;
    return r;
}

int pgcd(int a, int b) {                  // algorithme d'Euclide
    while (b != 0) { int r = a % b; a = b; b = r; }
    return a;
}

int ppcm(int a, int b) { return a / pgcd(a, b) * b; }

int est_premier(int n) {
    if (n < 2) return 0;
    for (int i = 2; i * i <= n; i++) if (n % i == 0) return 0;
    return 1;
}

long fibonacci(int n) {                   // F(0)=0, F(1)=1
    long a = 0, b = 1;
    for (int i = 0; i < n; i++) { long t = a + b; a = b; b = t; }
    return a;
}

int somme_chiffres(int n) {
    int s = 0;
    for (n = abs(n); n > 0; n /= 10) s += n % 10;
    return s;
}

int nb_chiffres(int n) {
    int c = 0;
    do { c++; n /= 10; } while (n != 0);
    return c;
}

int inverser_nombre(int n) {
    int r = 0;
    for (; n > 0; n /= 10) r = r * 10 + n % 10;
    return r;
}

int est_palindrome_nombre(int n) { return n == inverser_nombre(n); }

int est_parfait(int n) {                  // somme des diviseurs propres == n
    int s = 0;
    for (int i = 1; i <= n / 2; i++) if (n % i == 0) s += i;
    return s == n && n > 0;
}

int est_armstrong(int n) {                // ex : 153 = 1^3 + 5^3 + 3^3
    int k = nb_chiffres(n), s = 0;
    for (int m = n; m > 0; m /= 10) s += (int)pow(m % 10, k);
    return s == n;
}

void decimal_vers_binaire(int n) {        // affichage (récursif)
    if (n > 1) decimal_vers_binaire(n / 2);
    printf("%d", n % 2);
}

long factorielle_rec(int n) { return n <= 1 ? 1 : n * factorielle_rec(n - 1); }
```

F. Chaînes de caractères

```
int longueur(const char *s) {             // équivalent de strlen
    int i = 0;
    while (s[i] != '\0') i++;
    return i;
}

void inverser_chaine(char *s) {
    int n = strlen(s);
```

```
    for (int i = 0; i < n / 2; i++) { char t = s[i]; s[i] = s[n - 1 - i]; s[n - 1 - i] = t; }
}

int est_palindrome(const char *s) {
    int n = strlen(s);
    for (int i = 0; i < n / 2; i++) if (s[i] != s[n - 1 - i]) return 0;
    return 1;
}

int compter_voyelles(const char *s) {
    int c = 0;
    for (int i = 0; s[i]; i++)
        if (strchr("aeiouyAEIOUY", s[i])) c++;
    return c;
}

int compter_mots(const char *s) {
    int c = 0, dans_mot = 0;
    for (int i = 0; s[i]; i++) {
        if (!isspace((unsigned char)s[i]) && !dans_mot) { c++; dans_mot = 1; }
        else if (isspace((unsigned char)s[i])) dans_mot = 0;
    }
    return c;
}

void en_majuscules(char *s) {
    for (int i = 0; s[i]; i++) s[i] = toupper((unsigned char)s[i]);
}

void saisir_chaine(char *s, int taille) { // lecture d'une ligne avec espaces
    fgets(s, taille, stdin);
    s[strcspn(s, "\n")] = '\0';           // supprime le \n final
}

int compter_char(const char *s, char c) {
    int n = 0;
    for (int i = 0; s[i]; i++) if (s[i] == c) n++;
    return n;
}
```

G. Matrices (tableaux 2D)

```
void saisir_mat(int n, int m, int a[n][m]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
            printf("a[%d][%d] = ", i, j);
            scanf("%d", &a[i][j]);
        }
}

void afficher_mat(int n, int m, int a[n][m]) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) printf("%4d", a[i][j]);
        printf("\n");
    }
}

int somme_diagonale(int n, int a[n][n]) {
    int s = 0;
    for (int i = 0; i < n; i++) s += a[i][i];
    return s;
}

void transposee(int n, int m, int a[n][m], int t[m][n]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) t[j][i] = a[i][j];
}

void somme_mat(int n, int m, int a[n][m], int b[n][m], int c[n][m]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) c[i][j] = a[i][j] + b[i][j];
}

void produit_mat(int n, int p, int m, int a[n][p], int b[p][m], int c[n][m]) {
    for (int i = 0; i < n; i++)
        for (int j = 0; j < m; j++) {
```

```
            c[i][j] = 0;
            for (int k = 0; k < p; k++) c[i][j] += a[i][k] * b[k][j];
        }
}
```

H. Structures et tableaux de structures

```
typedef struct {
    char nom[30];
    int age;
    float moyenne;
} Etudiant;

void saisir_etudiant(Etudiant *e) {
    printf("Nom : ");      scanf("%29s", e->nom);
    printf("Age : ");      scanf("%d", &e->age);
    printf("Moyenne : ");  scanf("%f", &e->moyenne);
}

void afficher_etudiant(const Etudiant *e) {
    printf("%-15s %3d ans  moy = %.2f\n", e->nom, e->age, e->moyenne);
}

void saisir_etudiants(Etudiant tab[], int n) {
    for (int i = 0; i < n; i++) saisir_etudiant(&tab[i]);
}

void afficher_etudiants(const Etudiant tab[], int n) {
    for (int i = 0; i < n; i++) afficher_etudiant(&tab[i]);
}

int chercher_par_nom(const Etudiant tab[], int n, const char *nom) {
    for (int i = 0; i < n; i++) if (strcmp(tab[i].nom, nom) == 0) return i;
    return -1;
}

void trier_par_moyenne(Etudiant tab[], int n) {   // tri décroissant
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (tab[j].moyenne < tab[j + 1].moyenne) {
                Etudiant t = tab[j]; tab[j] = tab[j + 1]; tab[j + 1] = t;
            }
}

float moyenne_classe(const Etudiant tab[], int n) {
    float s = 0;
    for (int i = 0; i < n; i++) s += tab[i].moyenne;
    return s / n;
}
```

I. Allocation dynamique d'un tableau

```
int *creer_tab(int n) {
    int *t = malloc(n * sizeof(int));
    if (t == NULL) { printf("Erreur mémoire\n"); exit(1); }
    return t;
}

int **creer_mat(int n, int m) {           // matrice dynamique
    int **a = malloc(n * sizeof(int *));
    for (int i = 0; i < n; i++) a[i] = malloc(m * sizeof(int));
    return a;
}

void liberer_mat(int **a, int n) {
    for (int i = 0; i < n; i++) free(a[i]);
    free(a);
}
// usage : int *t = creer_tab(n); ... free(t);
```

J. Listes chaînées simples

```
typedef struct Noeud {
    int val;
    struct Noeud *suivant;
} Noeud;
```

```c
Noeud *creer_noeud(int x) {
    Noeud *n = malloc(sizeof(Noeud));
    n->val = x;  n->suivant = NULL;
    return n;
}

Noeud *inserer_debut(Noeud *tete, int x) {
    Noeud *n = creer_noeud(x);
    n->suivant = tete;
    return n;                           // nouvelle tête
}

Noeud *inserer_fin(Noeud *tete, int x) {
    Noeud *n = creer_noeud(x);
    if (tete == NULL) return n;
    Noeud *p = tete;
    while (p->suivant != NULL) p = p->suivant;
    p->suivant = n;
    return tete;
}

void afficher_liste(const Noeud *tete) {
    for (const Noeud *p = tete; p != NULL; p = p->suivant) printf("%d -> ", p->val);
    printf("NULL\n");
}

int taille_liste(const Noeud *tete) {
    int c = 0;
    for (; tete != NULL; tete = tete->suivant) c++;
    return c;
}

Noeud *rechercher_liste(Noeud *tete, int x) {
    while (tete != NULL && tete->val != x) tete = tete->suivant;
    return tete;                        // NULL si absent
}

Noeud *supprimer_valeur(Noeud *tete, int x) {
    Noeud *prec = NULL, *p = tete;
    while (p != NULL && p->val != x) { prec = p; p = p->suivant; }
    if (p == NULL) return tete;         // non trouvé
    if (prec == NULL) tete = p->suivant;  // suppression de la tête
    else prec->suivant = p->suivant;
    free(p);
    return tete;
}

Noeud *inverser_liste(Noeud *tete) {
    Noeud *prec = NULL, *suiv;
    while (tete != NULL) { suiv = tete->suivant; tete->suivant = prec; prec = tete; tete = suiv; }
    return prec;
}

void liberer_liste(Noeud *tete) {
    while (tete != NULL) { Noeud *t = tete; tete = tete->suivant; free(t); }
}
```

K. Piles et files (avec tableau)

```c
#define MAX 100
typedef struct { int data[MAX]; int sommet; } Pile;    // initialiser sommet = -1

int pile_vide(const Pile *p)  { return p->sommet == -1; }
int pile_pleine(const Pile *p) { return p->sommet == MAX - 1; }
void empiler(Pile *p, int x)  { if (!pile_pleine(p)) p->data[++p->sommet] = x; }
int depiler(Pile *p)          { return pile_vide(p) ? -1 : p->data[p->sommet--]; }

typedef struct { int data[MAX]; int debut, fin, nb; } File;  // tout à 0
void enfiler(File *f, int x) {
    if (f->nb < MAX) { f->data[f->fin] = x; f->fin = (f->fin + 1) % MAX; f->nb++; }
}
int defiler(File *f) {
    if (f->nb == 0) return -1;
    int x = f->data[f->debut];
    f->debut = (f->debut + 1) % MAX;  f->nb--;
    return x;
```

```
}
```

L. Fichiers

```
int ecrire_tab_fichier(const char *nom, const int t[], int n) {
    FILE *f = fopen(nom, "w");
    if (f == NULL) { perror("fopen"); return 0; }
    fprintf(f, "%d\n", n);
    for (int i = 0; i < n; i++) fprintf(f, "%d\n", t[i]);
    fclose(f);
    return 1;
}

int lire_tab_fichier(const char *nom, int t[], int max) {   // retourne n lu
    FILE *f = fopen(nom, "r");
    if (f == NULL) { perror("fopen"); return -1; }
    int n = 0;
    fscanf(f, "%d", &n);
    for (int i = 0; i < n && i < max; i++) fscanf(f, "%d", &t[i]);
    fclose(f);
    return n;
}

int compter_lignes(const char *nom) {
    FILE *f = fopen(nom, "r");
    if (f == NULL) return -1;
    int c = 0, ch;
    while ((ch = fgetc(f)) != EOF) if (ch == '\n') c++;
    fclose(f);
    return c;
}

void afficher_fichier(const char *nom) {
    FILE *f = fopen(nom, "r");
    if (f == NULL) { perror("fopen"); return; }
    char ligne[256];
    while (fgets(ligne, sizeof(ligne), f)) printf("%s", ligne);
    fclose(f);
}

void sauver_etudiants_bin(const char *nom, const Etudiant tab[], int n) {
    FILE *f = fopen(nom, "wb");
    if (f == NULL) return;
    fwrite(tab, sizeof(Etudiant), n, f);
    fclose(f);
}
```

PARTIE 3 - Modèle de programme et pièges

Modèle de programme avec menu

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define MAX 100

/* ... prototypes puis fonctions de la partie 2 ... */

int main(void) {
    int t[MAX], n, choix;
    srand(time(NULL));
    n = saisir_positif();
    saisir_tab(t, n);

    do {
        printf("\n1. Afficher  2. Somme  3. Max  4. Trier  0. Quitter\n> ");
        scanf("%d", &choix);
        switch (choix) {
            case 1: afficher_tab(t, n); break;
            case 2: printf("Somme = %d\n", somme_tab(t, n)); break;
            case 3: printf("Max = %d\n", max_tab(t, n)); break;
            case 4: tri_bulles(t, n); afficher_tab(t, n); break;
            case 0: printf("Au revoir\n"); break;
            default: printf("Choix invalide\n");
        }
    } while (choix != 0);
    return 0;
}
```

Bonnes pratiques et pièges classiques

- Toujours initialiser les variables (somme à 0, produit à 1, min/max au premier élément).
- Compiler avec `-Wall -Wextra` et corriger tous les avertissements.
- Vérifier le retour de `malloc`, `fopen`, `scanf`.
- Ne jamais utiliser `gets` (dangereux) : utiliser `fgets`.
- `=` affecte, `==` compare (erreur classique dans un `if`).
- Après `scanf("%d")`, un `\n` reste dans le buffer : appeler `vider_buffer()` avant `fgets`/`getchar`.
- Les indices vont de `0` à `n-1` : `for (i = 0; i < n; i++)`, pas `<= n`.
- Dans `scanf`, ne pas oublier `&` (sauf pour les tableaux/chaînes).
- Comparer des chaînes avec `strcmp`, copier avec `strcpy`/`strncpy`, jamais avec `=` ou `==`.
- Division entière : `(float)a / b` pour obtenir un réel.
- Ne jamais retourner l'adresse d'une variable locale.
- Libérer toute la mémoire allouée (`free`) et mettre le pointeur à `NULL`.
- Dans un `switch`, ne pas oublier `break`.
- Un tableau passé en paramètre : toujours transmettre aussi sa taille `n`.
- `float` dans `printf` : `%f` ; `double` dans `scanf` : `%lf`.